赛博密室·零点逃脱

一、你们要完成什么

用 C 语言实现一个从文件读取关卡和操作脚本的密室小游戏。玩家在地图上移动，调查线索取得开门密码，拾取钥匙，到密码门前执行开门操作，最后到达出口。游戏有行动次数限制；用完次数仍未逃脱，就算失败。

必须实现的核心功能是：读取并检查关卡；按脚本逐条执行移动、调查、拾取、开门；判断行动是否成功、是否消耗次数；判断逃脱或失败；按固定格式写出结果。只设一条线索、一把钥匙、一扇密码门和一个出口。

二、项目结构与统一运行方式

提交的项目文件夹统一命名为 `cyber_room` 。根目录必须有 `main.c` 和 `Makefile` ；其他 `.c` 、 `.h` 文件等根据实际情况设计。请附一份 `README.md` ，简要说明模块分工等信息。不要提交只有可执行文件而没有源代码的项目。

教师进入项目根目录后，使用：

```
make clean
make
./cyber_room --level level.txt --script script.txt --output result.txt
```

第三行是**运行命令，不是编译命令**。 `make` 负责调用 `gcc` 编译根目录下的全部 `.c` 文件，生成名为 `cyber_room` 的可执行文件。教师可能把三个文件参数换成其他路径，所以不要把输入、输出文件名写死在程序里；除这些命令行参数外，不得再要求用户手动输入。

下面的 `Makefile` 可以直接作为起点。如果你们把 `.c` 文件放进子目录，就需要自行修改 `SRC` ；如果全部放在根目录，通常不必改。命令行前的缩进必须是 **Tab**，不是空格。

```
CC = gcc
CFLAGS = -std=c11 -Wall -Wextra -pedantic -O2
TARGET = cyber_room
SRC = $(wildcard *.c)
OBJ = $(SRC:.c=.o)

.PHONY: all clean
all: $(TARGET)

$(TARGET): $(OBJ)
	$(CC) $(CFLAGS) -o $(TARGET) $(OBJ)

%.o: %.c
	$(CC) $(CFLAGS) -c $< -o $@

clean:
	rm -f $(TARGET) $(OBJ)
```

三、关卡文件 `level.txt`

关卡文件使用纯文本，格式固定：

```
SIZE 行数 列数
TURNS 最大行动次数
MAP
地图第1行
...
地图第N行
ENDMAP
```

例如：

```
SIZE 3 11
TURNS 20
MAP
###########
#@.C.K.D.E#
###########
ENDMAP
```

要求如下：

- 行数为 3～20，列数为 3～40；地图恰有指定的行数，每行恰有指定的列数。
- `TURNS` 为 1～100。
- 地图只允许 `#`、`.`、`@`、`C`、`K`、`D`、`E`。`#` 是墙，`.` 是空地，`@` 是玩家起点，`C` 是藏有开门密码的线索，`K` 是钥匙，`D` 是密码门，`E` 是出口。本项目不处理具体的密码数字；密码被简化为“是否已取得”的状态。只有调查 `C` 才能取得密码，关卡和脚本中都不另写密码。
- `@`、`C`、`K`、`D`、`E` 各恰好出现一次。地图坐标从左上角 `(0, 0)` 开始，先行后列。其他可通行格可为 `.`。
- 不要求检查地图是否一定存在逃脱路径。没有逃脱路径的合法关卡，也应能正常运行并给出失败结果。

四、脚本文件 `script.txt` 与游戏规则

脚本文件每行一条命令，空行忽略。支持以下命令，大小写按示例严格匹配：

命令	作用
`move up`、`move down`、`move left`、`move right`	向相应方向移动一格
`inspect`	在 `C` 格调查线索，取得开门密码
`take`	在 `K` 格拾取钥匙
`open`	站在 `D` 的上下左右相邻格， 使用已取得的密码和钥匙开门； 命令后不带参数

按文件顺序执行命令。玩家初始位于 `@`，已用行动数为 0，尚未取得密码、钥匙或打开门。最多执行 `TURNS` 次成功行动。

具体规则：

1. `move` 到地图外或墙上会失败，玩家位置不变，不消耗行动次数。未打开的 `D` 不能进入。正常进入可通行格消耗 1 次行动。
2. `inspect` 仅在玩家位于 `C` 且尚未调查时成功：从线索中取得开门密码，标记 `CLUE_FOUND=1`，消耗 1 次行动。在别处调查或重复调查，不改变状态、不消耗次数。**只有执行过这次成功的 `inspect`，玩家才拥有开门密码**；仅仅走到或经过 `C` 格不算取得。
3. `take` 仅在玩家位于 `K` 且尚未拾取时成功：标记“已取得钥匙”，消耗 1 次行动。在别处拾取或重复拾取，不改变状态、不消耗次数。

4. `open` 仅在玩家与 `D` 上下左右相邻、已通过 `inspect` 取得密码、已通过 `take` 取得钥匙、门尚未打开时成功：打开门，消耗 1 次行动。没有取得密码或钥匙、站得不够近、门已经打开时执行 `open`，都不改变状态、不消耗次数。程序不要求玩家输入或猜测密码，也不统计“输错密码”。
5. 门打开后，玩家才可以通过 `D`。玩家进入 `E` 时，若门已打开，立刻判定 `ESCAPED`；否则不能进入 `E`，也不消耗次数。
6. 每次消耗行动后检查结果：如果刚进入出口，优先判定 `ESCAPED`；否则，若已用行动数达到 `TURNS`，判定 `FAILED`。若脚本执行完还未逃脱，也判定 `FAILED`。达到最终结果后不再执行后续命令。
7. 空行以及不属于上述格式的命令不改变游戏状态、不消耗次数；程序继续读取下一行。可在终端提示无效命令，但不能因此破坏结果文件格式。

例如，玩家站在门旁并拿到钥匙，但还没有在 `C` 格执行 `inspect`，此时 `open` 不生效、不消耗行动次数。之后成功调查线索，再回到门旁执行 `open`，门才会打开。

五、正常结果文件格式

程序必须创建或覆盖 `--output` 指定的文件。正常完成时，文件**恰好**包含以下五行，字段名、顺序和空格保持一致，行末使用换行：

```
RESULT ESCAPED
ACTIONS 11
CLUE_FOUND 1
KEY_TAKEN 1
DOOR_OPENED 1
```

`RESULT` 只能是 `ESCAPED` 或 `FAILED`； `ACTIONS` 是已消耗的行动次数；接下来的三个字段是 0 或 1。其中 `CLUE_FOUND 1` 表示已通过 `inspect` 取得密码。不要在结果文件中写地图、过程日志、中文说明或其他字段。过程提示可以输出到终端。

六、一个可以直接照着测试的完整例子

把下面内容保存为 `level.txt`：

```
SIZE 3 11
TURNS 20
MAP
###########
#@.C.K.D.E#
###########
ENDMAP
```

把下面内容保存为 `script.txt`：

```
move right
move right
inspect
move right
move right
take
move right
open
move right
move right
move right
```

执行：

```
make
./cyber_room --level level.txt --script script.txt --output result.txt
```

执行过程：玩家从第 1 列移动到第 3 列调查线索并取得密码，再到第 5 列取钥匙，走到第 6 列执行 `open` 打开第 7 列的门，最后经过门进入第 9 列的出口。共消耗 11 次行动，因此 `result.txt` 应当**完全等于**：

```
RESULT ESCAPED
ACTIONS 11
CLUE_FOUND 1
KEY_TAKEN 1
DOOR_OPENED 1
```

再试着把 `TURNS 20` 改为 `TURNS 8`：第 8 次行动成功打开门后，次数达到上限，但尚未进入出口，结果应为 `FAILED`，`ACTIONS 8`，其他状态仍按当时实际值输出。这个例子用来检查你们对行动次数和结束条件的实现。